

FIAP
DEVOPS TOOLS & CLOUD COMPUTING

OLLI PET / CLYVO CHALLENGE

Containerização em Nuvem de API Java Spring Boot com Docker e H2

Henrique Rodrigues Vespasiano — RM562917

Ruan Luca Feliciano — RM562218

Andre Rosa Colombo — RM563112

Mirelly Sousa Alves — RM566299

Gabriely Bonfim Silva — RM566242

São Paulo

2026

SUMÁRIO

1 INTRODUÇÃO.....	3
2 DESCRIÇÃO DO PROJETO.....	3
3 BENEFÍCIOS PARA O NEGÓCIO.....	4
4 ARQUITETURA MACRO DA SOLUÇÃO	4
5 TECNOLOGIAS UTILIZADAS	5
6 REQUISITOS ATENDIDOS	5
7 EXECUÇÃO EM NUVEM COM AZURE	5
8 CONTEINERIZAÇÃO COM DOCKER.....	7
9 BANCO DE DADOS H2 E PERSISTÊNCIA	7
10 TESTES DA API E CRUD	9
11 LINKS DA ENTREGA.....	13
12 REMOÇÃO DOS RECURSOS EM NUVEM.....	13
13 CONCLUSÃO.....	13
REFERÊNCIAS	14

1 INTRODUÇÃO

Este documento apresenta a entrega da disciplina DevOps Tools & Cloud Computing, referente à containerização e execução em nuvem de uma aplicação Java Spring Boot. O projeto escolhido foi o Olli Pet / Clyvo Challenge, uma API voltada para uma plataforma de intermediação veterinária inteligente.

O objetivo da entrega foi disponibilizar a aplicação em uma máquina virtual Linux na Microsoft Azure, utilizando Docker para containerizar a API e o banco de dados H2. Também foram validados os requisitos de persistência de dados, execução em background, acesso externo, testes via Swagger, testes via cURL e documentação das evidências.

A documentação foi reorganizada em padrão acadêmico, com seções numeradas, sumário automático, tabelas, legendas e prints posicionados de forma limpa, evitando sobreposição ou imagens desalinhadas.

2 DESCRIÇÃO DO PROJETO

O projeto Olli Pet / Clyvo Challenge representa a base operacional de uma plataforma veterinária inteligente. A aplicação centraliza informações de responsáveis, pets, veterinários e prontuários, permitindo o gerenciamento dos principais dados envolvidos em atendimentos veterinários.

A proposta de negócio vai além de um CRUD simples. A visão futura é preparar a Clivovet/Clyvo para atuar como um cérebro operacional veterinário, com IA multiagente, triagem por sintomas em linguagem natural, perfil comportamental contínuo do pet, recomendação de clínicas por localização, especialidade, fila e custo, além de integração com dispositivos IoT, como coleiras e sensores.

Nesta sprint, o foco técnico foi garantir que a API estivesse funcional, containerizada, documentada e executando em ambiente de nuvem, respeitando os requisitos definidos para a disciplina.

3 BENEFÍCIOS PARA O NEGÓCIO

A solução traz benefícios diretos para a Clivovet/Clyvo, pois centraliza dados de tutores, pets, veterinários e prontuários. Com essas informações organizadas, a empresa ganha

Java 21 e Spring Boot	Desenvolvimento da API principal.
Spring Data JPA	Camada de persistência e integração com entidades.
H2 Database	Banco de dados utilizado para a entrega e validação de persistência.
Docker	Containerização da API e do banco.
Docker Compose	Orquestração dos containers da aplicação e do banco.
Azure CLI	Criação e automação da infraestrutura em nuvem.
VM Linux Ubuntu	Ambiente de execução em nuvem.
Swagger/OpenAPI	Documentação e testes das rotas da API.
Git e GitHub	Versionamento e disponibilização do projeto.

6 REQUISITOS ATENDIDOS

A tabela a seguir apresenta os principais requisitos da atividade e o status de atendimento no projeto.

Requisito	Status / Evidência
CRUD com GET, POST, PUT e DELETE	Atendido e validado no Swagger e no script cURL.
Pelo menos 2 inserts significativos	Atendido com registros de responsáveis, pets e veterinários.
Persistência com H2 ou Oracle	Atendido com H2 em container.
Banco containerizado	Atendido com container clyvo-h2-db.
Dockerfile e Docker Compose	Atendido no repositório do projeto.
Volume nomeado para persistência	Atendido com clyvo-h2-data.
Aplicação em background	Atendido com docker compose up -d.
Aplicação sem usuário root	Atendido com usuário appuser.
VM Linux na Azure	Atendido com VM Ubuntu.
Portas necessárias abertas	Atendido: 22, 8080 e 8082.
Script Azure CLI	Atendido para criação e remoção dos recursos.
README no GitHub	Atendido no repositório.
Arquitetura macro	Atendido nesta documentação.
Vídeo no YouTube	A preencher após gravação.
Print de exclusão da VM	A executar somente após finalizar testes e vídeo.

7 EXECUÇÃO EM NUVEM COM AZURE

A infraestrutura foi criada na Azure por meio de script Azure CLI. O processo contempla a criação do Resource Group, o provisionamento de uma VM Linux Ubuntu, a abertura das portas necessárias, a instalação de Docker, Docker Compose, Git, Nano e JQ, o clone do repositório e a execução da solução com Docker Compose.

A VM recebeu um IP público utilizado para acessar a aplicação externamente. As evidências a seguir comprovam a criação da VM e a conectividade das portas exigidas.

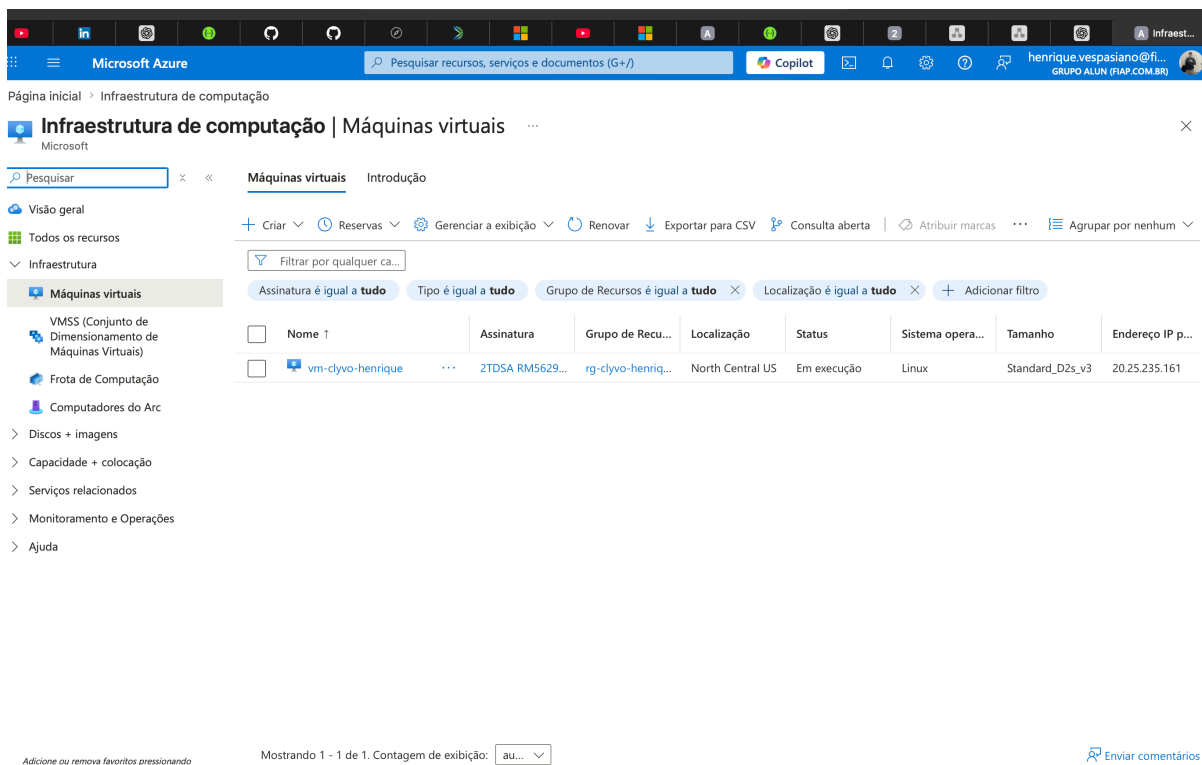


Figura 2 – Máquina virtual criada na Azure com IP público configurado.

```
nc -vz 20.25.235.161 22
nc -vz 20.25.235.161 8080
nc -vz 20.25.235.161 8082
```

```
Last login: Tue May 19 19:17:41 on ttys000

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBook-Air-de-Henrique:~ riquezada$ nc -vz 20.25.235.161 22
Connection to 20.25.235.161 port 22 [tcp/ssh] succeeded!
MacBook-Air-de-Henrique:~ riquezada$ nc -vz 20.25.235.161 8080
Connection to 20.25.235.161 port 8080 [tcp/http-alt] succeeded!
MacBook-Air-de-Henrique:~ riquezada$ nc -vz 20.25.235.161 8082
Connection to 20.25.235.161 port 8082 [tcp/us-cli] succeeded!
MacBook-Air-de-Henrique:~ riquezada$
```

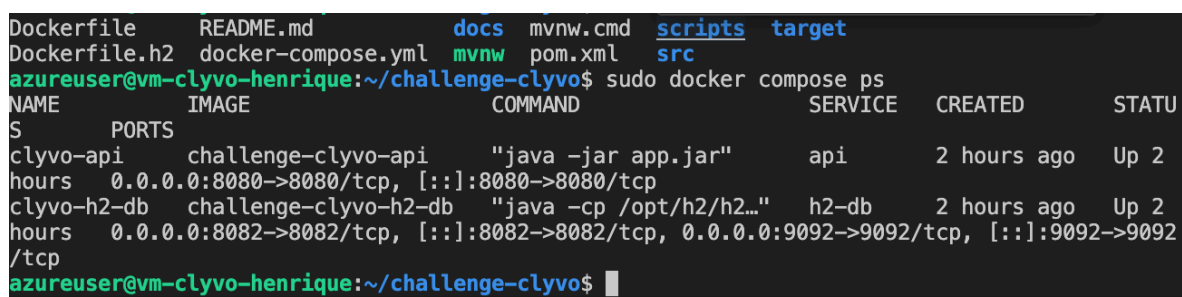
Figura 3 – Teste de conectividade das portas 22, 8080 e 8082.

8 CONTEINERIZAÇÃO COM DOCKER

A aplicação foi containerizada utilizando Docker. O projeto possui um Dockerfile para a API Java Spring Boot e um Dockerfile específico para o banco H2. A orquestração dos containers foi feita com Docker Compose, permitindo subir a API e o banco com um único comando.

A solução foi executada em background, atendendo ao requisito de manter o projeto rodando na VM sem depender de terminal aberto.

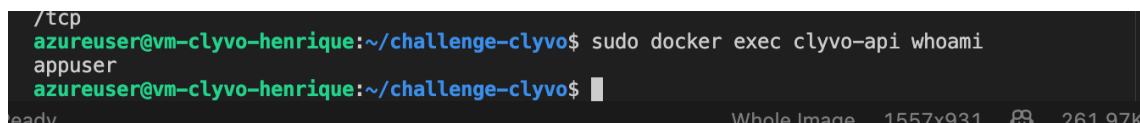
```
sudo docker compose ps
```



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
clyvo-api	challenge-clyvo-api	"java -jar app.jar"	api	2 hours ago	Up 2
clyvo-h2-db	challenge-clyvo-h2-db	"java -cp /opt/h2/h2..."	h2-db	2 hours ago	Up 2

Figura 4 – Containers clyvo-api e clyvo-h2-db em execução com status Up.

```
sudo docker exec clyvo-api whoami
```



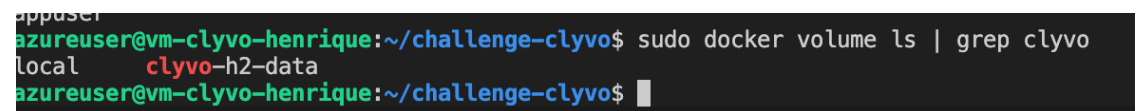
```
appuser
```

Figura 5 – Validação do usuário appuser no container da API.

9 BANCO DE DADOS H2 E PERSISTÊNCIA

O banco escolhido para a entrega foi o H2 Database. Ele foi executado em um container separado da aplicação, permitindo que a API se conecte ao banco via TCP. Para garantir persistência, foi configurado o volume nomeado clyvo-h2-data no Docker Compose.

```
sudo docker volume ls | grep clyvo
```



```
local clyvo-h2-data
```

Figura 6 – Volume Docker nomeado clyvo-h2-data utilizado para persistência.

O H2 Console também foi acessado pelo navegador, comprovando a conexão com o banco e a existência das tabelas principais da aplicação.

The screenshot shows the H2 Console interface. On the left, a sidebar lists the database schema: MED_VET, PET, PRONTUARIO, RESPONSAVEL, and INFORMATION_SCHEMA. The main area displays the results of SQL queries. The first query, 'SHOW TABLES;', returns a list of four tables: MED_VET, PET, PRONTUARIO, and RESPONSAVEL, all in the PUBLIC schema. The second query, 'SELECT * FROM RESPONSAVEL;', returns a table with four columns: ID, CPF, EMAIL, NOME, and DATA_NASC. It contains four rows of data, including a record for 'Teste Persistencia Azure'. The third query, 'SELECT * FROM PET;', returns a table with six columns: ID_PET, DATA_NASC, DESCRICAO, NOME, RACA, and ID_RESP. It contains two rows of data, including a record for 'Gata idosa acompanhada por triagem inteligente para risco renal'.

Figura 7 – H2 Console exibindo tabelas e registros persistidos no banco.

A persistência foi validada criando um registro, reiniciando os containers e consultando novamente os dados. Após o restart, o registro “Teste Persistencia Azure” permaneceu disponível, demonstrando que os dados foram mantidos no volume Docker.

```
sudo docker compose restart
sudo docker compose ps
curl http://20.25.235.161:8080/responsaveis
```

```
azureuser@vm-clyvo-henrique:~/challenge-clyvo$ sudo docker compose restart
[+] restart 0/2
  ✨ Container clyvo-api Restarting 1.2s
  ✨ Container clyvo-h2-db Restarting 1.2s
azureuser@vm-clyvo-henrique:~/challenge-clyvo$ sudo docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
S                    PORTS
clyvo-api            challenge-clyvo-api                 "java -jar app.jar"                  api        3 hours ago   Up 7
seconds             0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
clyvo-h2-db          challenge-clyvo-h2-db               "java -cp /opt/h2/h2..."           h2-db      3 hours ago   Up 7
seconds             0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp, 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
```

Figura 8 – Reinicialização dos containers e confirmação do status Up após restart.

```

azureuser@vm-clyvo-henrique:~/challenge-clyvo$ curl http://20.25.235.161:8080/responsaveis
{"content":[{"id":3,"nome":"Beatriz Lima","email":"beatriz.lima@email.com","cpf":"56789012345","dataNascimento":"1998-06-12"},{"id":2,"nome":"Joao Pedro Alves","email":"joao.alves@email.com","cpf":"23456789012","dataNascimento":"1992-04-22"},{"id":1,"nome":"Maria Fernanda Silva","email":"maria.fernanda@email.com","cpf":"12345678901","dataNascimento":"1995-08-10"},{"id":4,"nome":"Teste Persistencia Azure","email":"persistencia.azure@email.com","cpf":"99988877766","dataNascimento":"2000-01-01"}],"pageable":{"pageNumber":0,"pageSize":10,"sort":{"sorted":true,"empty":false,"unsorted":false},"offset":0,"paged":true,"unpaged":false},"totalElements":4,"totalPages":1,"last":true,"size":10,"number":0,"sort":{"sorted":true,"empty":false,"unsorted":false},"numberOfElements":4,"first":true,"empty":false}
azureuser@vm-clyvo-henrique:~/challenge-clyvo$

```

Figura 9 – Consulta após restart retornando o registro “Teste Persistencia Azure”.

10 TESTES DA API E CRUD

A API possui rotas CRUD para responsáveis, pets, veterinários e prontuários. Os testes foram realizados pelo IP público da VM, comprovando que a aplicação estava acessível pela internet e funcionando corretamente em ambiente de nuvem.

O Swagger apresenta os métodos GET, POST, PUT e DELETE das entidades, permitindo validar visualmente a documentação dos endpoints.

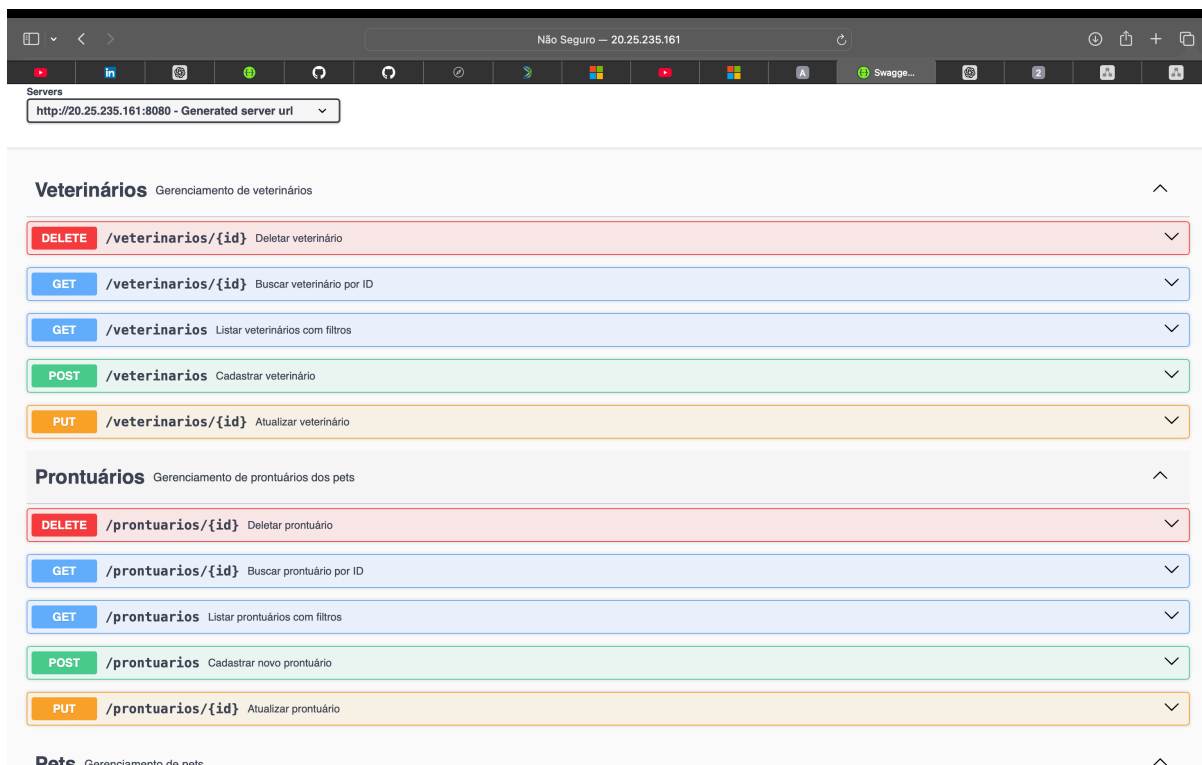


Figura 10 – Swagger exibindo rotas CRUD de veterinários e prontuários.

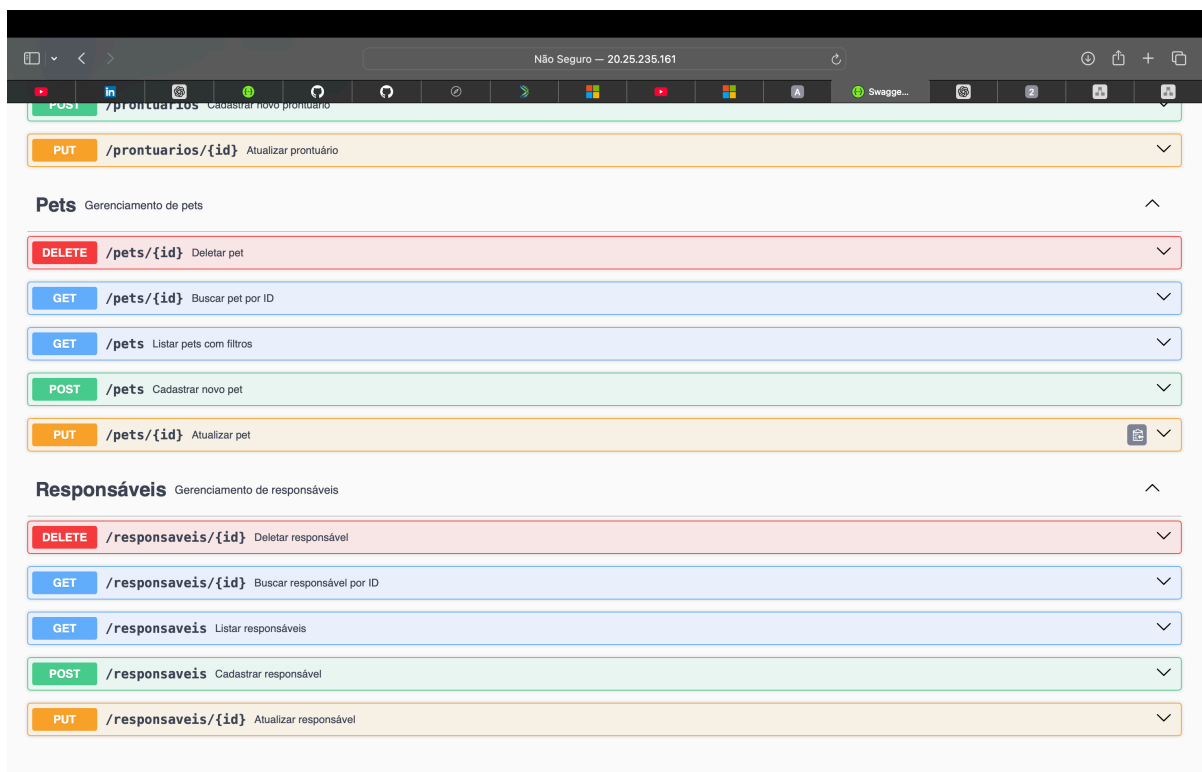


Figura 11 – Swagger exibindo rotas CRUD de pets e responsáveis.

Além dos testes pelo Swagger, foi executado um script automatizado com cURL para validar GET, POST, PUT e DELETE. A exclusão retornou HTTP 204, conforme solicitado no enunciado.

```
./scripts/testes-api-curl.sh http://20.25.235.161:8080
```

```
.161:8080lenge-clyvo$ ./scripts/testes-api-curl.sh http://20.25.235.161:8080
1) Listar responsáveis
{
  "content": [
    {
      "id": 3,
      "nome": "Beatriz Lima",
      "email": "beatriz.lima@email.com",
      "cpf": "56789012345",
      "dataNascimento": "1998-06-12"
    },
    {
      "id": 2,
      "nome": "Joao Pedro Alves",
      "email": "joao.alves@email.com",
      "cpf": "23456789012",
      "dataNascimento": "1992-04-22"
    },
    {
      "id": 1,
      "nome": "Maria Fernanda Silva",
      "email": "maria.fernanda@email.com",
      "cpf": "12345678901",
      "dataNascimento": "1995-08-10"
    },
    {
      "id": 4,
      "nome": "Teste Persistencia Azure",
      "email": "persistencia.azure@email.com",
      "cpf": "99988877766",
      "dataNascimento": "2000-01-01"
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 10,
    "sort": {
      "sorted": true,
      "empty": false,

```

Figura 12 – Script cURL listando responsáveis cadastrados por meio do endpoint GET.

```

"pageable": {
  "pageNumber": 0,
  "pageSize": 10,
  "sort": {
    "sorted": true,
    "empty": false,
    "unsorted": false
  },
  "offset": 0,
  "paged": true,
  "unpaged": false
},
"totalElements": 2,
"totalPages": 1,
"last": true,
"size": 10,
"number": 0,
"sort": {
  "sorted": true,
  "empty": false,
  "unsorted": false
},
"numberOfElements": 2,
"first": true,
"empty": false
}
4) Criar pet
{
  "id": 3,
  "nome": "Luna",
  "descricao": "Pet com monitoramento preventivo por perfil comportamental",
  "raca": "SRD",
  "dataNascimento": "2022-01-20",
  "responsavelId": 1,
  "nomeResponsavel": "Maria Fernanda Silva"
}
5) Atualizar pet 1
{
  "id": 1,
  "nome": "Thor",
  "descricao": "Cachorro acompanhado pela triagem inteligente da Clyvo",
  "raca": "Golden Retriever",
  "dataNascimento": "2021-03-15",
  "responsavelId": 1,
  "nomeResponsavel": "Maria Fernanda Silva"
}

```

Figura 13 – Script cURL validando criação e atualização de pet com POST e PUT.

```

}
6) Deletar pet 3, caso exista
HTTP/1.1 204
Date: Fri, 22 May 2026 18:07:51 GMT

7) Acessos úteis
http://20.25.235.161:8080/swagger-ui.html
http://20.25.235.161:8080/h2-console

```

Figura 14 – Script cURL validando DELETE com retorno HTTP 204.

Também foi feita uma chamada externa direta para o endpoint de responsáveis, retornando JSON com os dados cadastrados.

curl http://20.25.235.161:8080/responsaveis

```

http://20.25.235.161:8080/h2-console
azureuser@vm-clyvo-henrique:~/challenge-clyvo$ curl http://20.25.235.161:8080/responsaveis
{"content":[{"id":3,"nome":"Beatriz Lima","email":"beatriz.lima@email.com","cpf":"56789012345","dataNascimento":"1998-06-12"},{"id":2,"nome":"Joao Pedro Alves","email":"joao.alves@email.com","cpf":"23456789012","dataNascimento":"1992-04-22"},{"id":1,"nome":"Maria Fernanda Silva","email":"maria.fernanda@email.com","cpf":"12345678901","dataNascimento":"1995-08-10"},{"id":4,"nome":"Teste Persistencia Azure","email":"persistencia.azure@email.com","cpf":"99988877766","dataNascimento":"2000-01-01"}],"pageable":{"pageNumber":0,"pageSize":10,"sort":{"sorted":true,"empty":false,"unsorted":false},"offset":0,"paged":true,"unpaged":false},"totalElements":4,"totalPages":1,"last":true,"size":10,"number":0,"sort":{"sorted":true,"empty":false,"unsorted":false},"numberOfElements":4,"first":true,"empty":false}
azureuser@vm-clyvo-henrique:~/challenge-clyvo$

```

Figura 15 – cURL externo retornando JSON com responsáveis cadastrados.

11 LINKS DA ENTREGA

Os links abaixo devem ser utilizados para avaliação da entrega.

Item	Link
Repositório GitHub	https://github.com/HenriqueRodriguesV/challenge_clyvo-master
Vídeo no YouTube	COLE AQUI O LINK DO VIDEO

O link do vídeo deve ser preenchido somente após a gravação e publicação no YouTube.

12 REMOÇÃO DOS RECURSOS EM NUVEM

Após a conclusão dos testes, gravação do vídeo e coleta das evidências, os recursos criados na Azure devem ser removidos para evitar consumo desnecessário da assinatura. A remoção é feita por meio do script `azure-delete-resources.sh`, que exclui o Resource Group utilizado na entrega.

```
chmod +x scripts/azure-delete-resources.sh
./scripts/azure-delete-resources.sh
```

Atenção: esta etapa deve ser executada somente depois de gravar o vídeo e coletar todos os prints da aplicação funcionando.

ESPAÇO PARA PRINT FINAL — REMOÇÃO DOS RECURSOS EM NUVEM
Inserir aqui o print do terminal executando o script de remoção dos recursos em nuvem e/ou o print do Portal Azure mostrando que o Resource Group/VM foi removido.

13 CONCLUSÃO

A entrega demonstrou a containerização e execução em nuvem de uma API Java Spring Boot com banco H2. O projeto foi executado em uma VM Linux na Azure, utilizando Docker e Docker Compose para subir a aplicação e o banco em containers separados.

A solução atendeu aos requisitos da atividade, incluindo CRUD completo, inserts significativos, persistência com volume nomeado, execução em background, aplicação rodando

com usuário sem privilégios administrativos, scripts Azure CLI, documentação no GitHub, Swagger e validação externa da API.

Além da parte técnica, o projeto possui uma proposta de negócio coerente com o desafio da Clyvo/Olli Pet, servindo como base para uma futura plataforma veterinária inteligente com IA, automação e monitoramento preventivo.

REFERÊNCIAS

FIAP. Conteúdo acadêmico da disciplina DevOps Tools & Cloud Computing. São Paulo: FIAP, 2026.

MICROSOFT. Azure CLI Documentation. Disponível em: <https://learn.microsoft.com/cli/azure/>. Acesso em: 22 maio 2026.

DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: 22 maio 2026.

SPRING. Spring Boot Reference Documentation. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 22 maio 2026.

H2 DATABASE. H2 Database Engine Documentation. Disponível em: <https://www.h2database.com/>. Acesso em: 22 maio 2026.

GITHUB. Repositório do projeto Olli Pet / Clyvo Challenge. Disponível em: https://github.com/HenriqueRodriguesV/challenge_clyvo-master. Acesso em: 22 maio 2026.